

Evolution of Programming Languages

Evolution of programming languages refers to the development and improvement of programming languages from machine language to modern intelligent languages.

Programming languages evolved to make programming easier, faster, and more efficient.

Generations of Programming Languages

1. First Generation Language (1GL) – Machine Language

Machine language is the oldest programming language written in binary form (0s and 1s).

Features

- Directly understood by CPU
- Very fast execution
- Machine dependent
- Difficult to understand

Example

10101010 00011101

2. Second Generation Language (2GL) – Assembly Language

Assembly language uses mnemonic codes instead of binary instructions.

Features

- Easier than machine language
- Hardware-oriented
- Machine dependent

Example

MOV A, B

ADD A, C

3. Third Generation Language (3GL) – High-Level Language

High-level languages use English-like statements and are machine independent.

Features

- Easy to learn
- Faster program development
- Uses compiler/interpreter

Examples

- FORTRAN
- C
- Java
- Python

4. Fourth Generation Language (4GL) – Very High-Level Language

4GL focuses on what result is needed instead of how to achieve it.

Features

- Less code required
- Problem-oriented
- High productivity

Examples

- SQL
- MATLAB
- SAS

5. Fifth Generation Language (5GL) – AI-Oriented Language

5GL languages are used for Artificial Intelligence and logic-based systems.

Features

- Logic-based programming
- Used in expert systems
- Research-oriented

Example

- Prolog

Advantages of Modern Programming Languages

- Easy to understand
- Faster software development
- Better problem solving
- Supports advanced technologies

Describing Syntax in Programming Languages

Syntax in programming languages refers to the set of rules that define how programs should be written correctly.

Syntax specifies the correct structure of statements, expressions, keywords, operators, and symbols in a programming language.

Need for Describing Syntax

Syntax description is important because:

- Programmers can write correct programs
 - Compilers can understand programs
 - Syntax errors can be detected
 - Languages can be implemented properly
-

Methods Used to Describe Syntax

There are three common methods used to describe syntax.

1. Context-Free Grammar (CFG)

CFG is a formal method used to describe syntax using production rules.

Features

- Shows how expressions and statements are formed
- Used in parsing process

Example

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$

$F \rightarrow (E) \mid \text{id}$

Explanation

- E represents expression
 - T represents term
 - F represents factor
-

2. Backus–Naur Form (BNF)

BNF is a notation used to represent grammar rules clearly.

- Uses angle brackets $< >$
- $::=$ means “is defined as”

Example

$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$

3. Extended Backus–Naur Form (EBNF)

Definition

EBNF is an improved version of BNF with extra symbols for simplification.

Special Symbols

- $\{ \}$ → repetition
- $[]$ → optional part
- $()$ → grouping

Example

$\langle \text{stmt} \rangle ::= \text{"if"} \langle \text{expr} \rangle \text{"then"} \langle \text{stmt} \rangle [\text{"else"} \langle \text{stmt} \rangle]$

Advantages of Formal Syntax Description

- Precise and unambiguous
- Helps compiler designers
- Supports parser construction
- Detects syntax errors
- Standardizes programming languages

Describing Semantics in Programming Languages

- Semantics in programming languages refers to the meaning of program statements and expressions. It explains what a program actually does during execution.
 - Even if a program has correct syntax, it must also have correct semantics to produce meaningful output.
-

Need for Describing Semantics

Semantics is important because it:

- Defines meaning of statements
 - Helps understand program behavior
 - Detects semantic errors
 - Helps compiler design
 - Improves program correctness
-

Types of Semantics

There are two main methods used to describe semantics.

1. Operational Semantics

Operational semantics describes the meaning of a program by showing step-by-step execution.

Example

$A = A + 1$

The value of **A** increases by 1 after execution.

Advantages of Operational Semantics

- Easy to understand
 - Shows actual execution process
-

2. Denotational Semantics

Denotational semantics describes meaning using mathematical functions. Each program statement is mapped to a mathematical representation.

Advantages of Denotational Semantics

- Precise meaning
 - Useful for formal analysis
-

3. Axiomatic Semantics

Axiomatic semantics describes program behavior using logical rules and assertions. It checks whether a program satisfies certain conditions.

Example

Used to prove program correctness.

Advantages of Axiomatic Semantics

- Helps verify correctness
 - Useful in software testing
-

Semantic Errors

Semantic errors occur when the meaning of the program is incorrect even if syntax is correct.

Examples

- Type mismatch
- Using undeclared variables
- Invalid operations

Example

```
int x;  
x = "hello";
```

Here, string value cannot be assigned to an integer variable.

Advantages of Semantics

- Improves program reliability
- Helps compiler construction
- Detects logical mistakes
- Ensures correct program behavior

Difference Between Syntax and Semantics

S.No	Syntax	Semantics
1	Defines structure of program	Defines meaning of program
2	Checks grammar rules	Checks logical meaning
3	Detected during parsing	Detected during semantic analysis
4	Example: missing semicolon	Example: type mismatch

Parsing, Top-Down Parsing, and Bottom-Up Parsing

Parsing

- Parsing is the process of analyzing a sequence of tokens to check whether it follows the grammar rules of a programming language.
- Before parsing, a program is divided into small units called tokens such as keywords, identifiers, operators, and numbers.
- The parser checks the syntax of these tokens and builds a parse tree or syntax tree.
- Parsing helps the compiler understand the structure of a program.

Need for Parsing

Parsing is important because it:

- Detects syntax errors
- Checks grammar rules
- Builds parse tree
- Organizes statements and expressions
- Helps semantic analysis and code generation

Grammar Used in Parsing

Parsing is based on Context-Free Grammar (CFG).

Components of Grammar

- Terminals → actual symbols like `if`, `+`, `id`
- Non-terminals → variables like `E`, `T`
- Productions → grammar rules like `E → E + T`
- Start Symbol → starting point of grammar

Types of Parsing

There are two major parsing methods:

1. Top-Down Parsing
2. Bottom-Up Parsing

Top-Down Parsing

Top-down parsing starts from the start symbol and tries to derive the input string step by step.

Working

- Starts from start symbol
- Expands grammar rules
- Matches input tokens
- Builds parse tree from root to leaves

Common Top-Down Methods

1. **Recursive-Descent Parsing** - Uses recursive functions

2. **Predictive / LL Parsing**

- Reads input from Left to right
 - Produces Leftmost derivation
 - Uses parsing table
-

Bottom-Up Parsing

Bottom-up parsing starts from input symbols and reduces them to the start symbol.

Working

- Uses stack to store symbols
- Combines symbols step by step
- Builds parse tree from leaves to root

This method is also called Shift-Reduce Parsing.

Common Bottom-Up Methods

1. Shift-Reduce Parsing

- Shift → moves input symbol to stack
- Reduce → replaces symbols using grammar rule

2. LR Parsing

- Reads input Left to right
- Produces rightmost derivation in reverse
- Can handle most programming language grammars

Types of LR Parsing

- SLR
- Canonical LR
- LALR

Advantages of Parsing

- Detects syntax errors
- Checks grammar correctness
- Builds parse tree
- Shows program structure